Daniel Borkovec: 2941124B
Sara Novakova: 2917409N

Digital Signal Processing - Fourier Transform Report

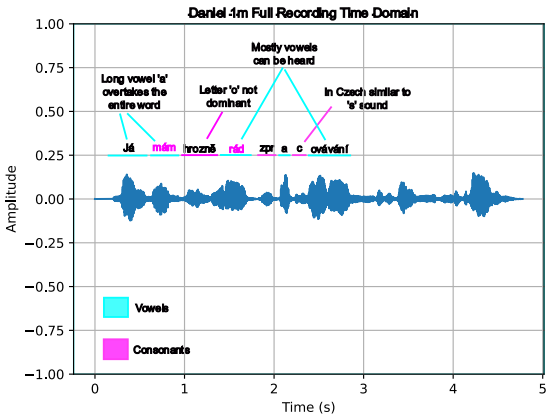
Introduction and Objectives

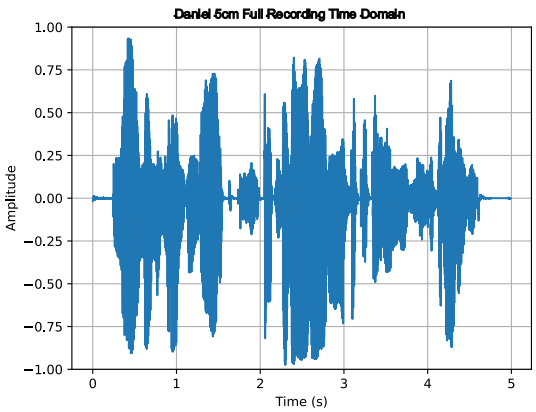
This report covers the first assignment of the DSP course, the digital, non-real time voice enhancement using the Fast Fourier Transform, hereafter referred to as FFT. The assignment is split into three distinct sections. The objective is to filter unwanted noise from the audio and improve the perceived prominence and clarity of the recorded speech.

First, two recorded voice samples at 48 kHz sampling rate, one at a close distance of 5 cm and one further away from the microphone at 1 m, were plotted in both the time domain and the frequency domain using an FFT. Secondly, the gains of specific FFT bins were altered to either attenuate or amplify certain frequencies of both recordings to enhance the voice in the audio. Lastly, a non-linear method of voice alteration which adds additional harmonics to the audio, an aural exciter, was used for enhancement as well, with the methodology behind its implementation documented and its results analyzed.

Part 1 - Plotting and Basic Analysis

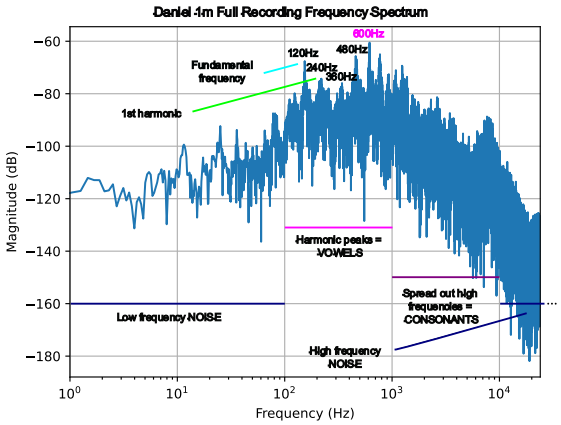
Initially, the first section builds the core plotting logic used in the subsequent sections. Using the Python library SciPy, the original 48 kHz sampling rate, 16-bit .wav files are converted into arrays for easier data manipulation. For normalization in the time domain, all values are divided by 32768.0, or 2^{15}. This is because a 16-bit audio recording ranges from - 2^{15} to 2^{15} - 1, meaning dividing by 32768.0, where keeping the trailing decimal ensures the result is a float. This normalizes the audio to a range from 1 to -1. The time-domain plots of both the 5 cm and the 1 m recordings of Daniel's voice can be seen below.

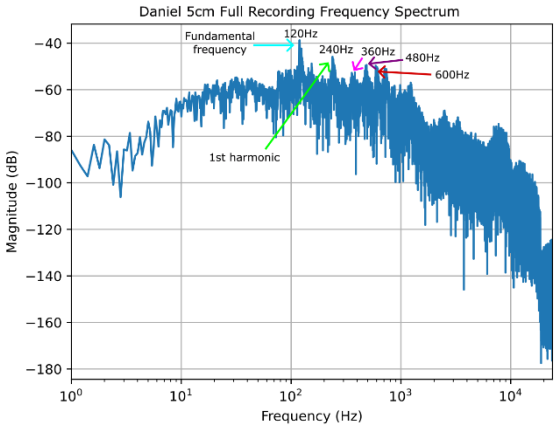




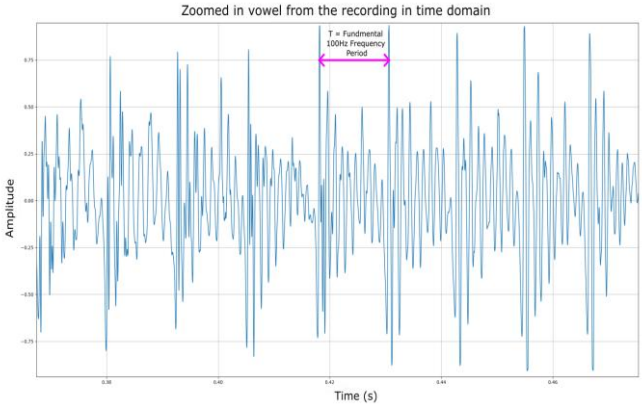
The 1m time-domain plot markings illustrate what parts of the recording correspond to which Czech words. Furthermore, when comparing the two plots, it is visible that the 1m recording is much quieter than the 5cm recording, as the highest amplitude with reference to the highest bit number is about 5x lower. Lastly, although the 5 cm is louder, the recording does not clip, as the highest value does not reach 1, indicating the microphone does not undergo saturation.

Subsequently, the Numpy library's FFT function was used to generate a frequency domain representation of the two recordings. As the Numpy FFT function returns the frequency bins on the x-axis, additional mathematics were required to turn the x-axis back into frequency. Unlike the data later used in the linear and non-linear enhancement pipeline, the time-domain recordings used only for visual inspection were passed through a Hamming window before the FFT was applied, which attenuates the beginning and end of the waveform, removing any potential higher frequencies in the FFT due to abrupt edges, leading to a more accurate spectrum.

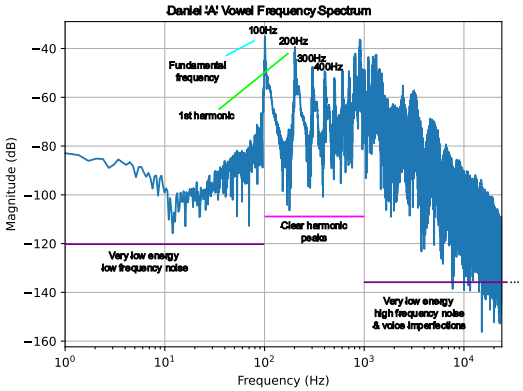


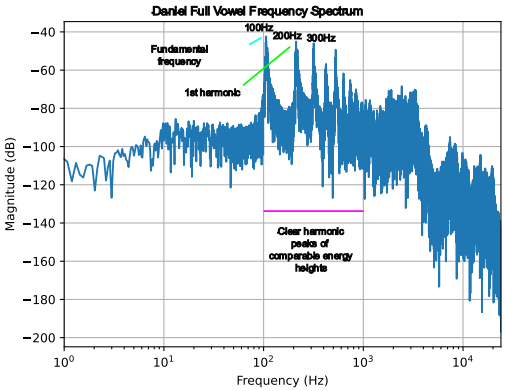


The frequency domain representation of Daniel's voice indicates a base harmonic of around 120 Hz, and subsequent harmonics at 240 Hz, 360 Hz, etc. In addition, to analyze the frequency regions of consonants and vowels, we inspected the time-domain recordings and matched the time to the spoken vowel or consonant. Below is a time-domain plot heavily zoomed in on the Czech equivalent of the letter "A":



The plot illustrates clearly that the region is heavily dominated by a fundamental frequency of around ~120 Hz for Daniel's voice. Furthermore, the periodic vocal cord vibrations in our throats not only produce harmonics alongside the fundamental frequency, as visible in the full spectrum plot, but also other higher frequency components. These higher frequency components are clearly visible in the zoomed in plot above, manifesting themselves as the high-frequency low-amplitude components in between the labeled peaks of the fundamental frequency.

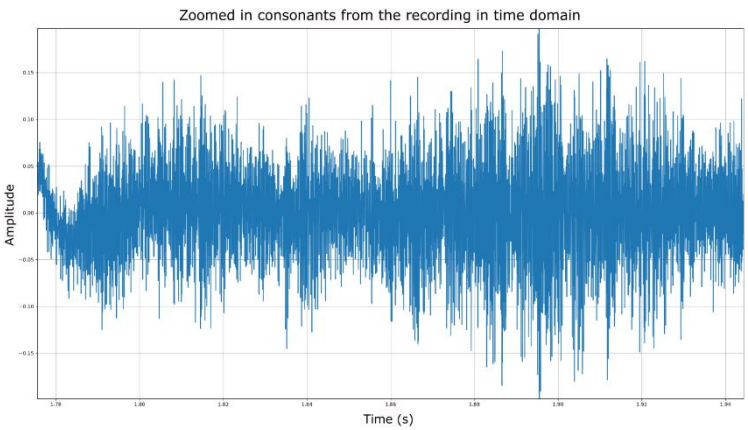


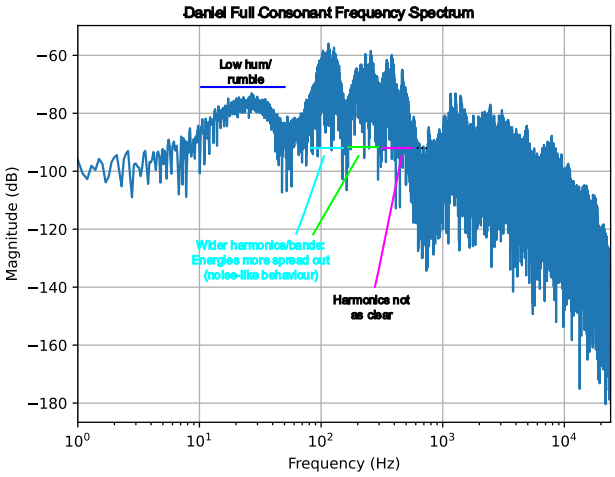


Moreover, after analyzing frequency plots of only the 'A' vowel, and comparing with a recording with all vowels, we observed that the all vowels spectrum also contained the ~100 Hz fundamental and its harmonics. However, the energy distribution between the harmonics was different, as is visible on the spectrum as peaks of varying amplitudes. This is expected, as each vowel is created by changing the shape of the mouth and throat, which creates filtering effects, attenuating certain harmonics' energy, and in turn creating a different vowel. Unlike the 'A' vowel spectrum, which features greater differences in energy levels between the different harmonics, the frequency spectrum of all core vowels shows a more comparable amplitude between the various harmonics. This is likely because of the different energy distributions of the core vowels combined.

Thus, we concluded that the vowels are periodic (harmonic) and predominantly located in the 100 Hz – 2 kHz, for the energy of the harmonic's decays rapidly after 1 kHz and sharp peaks are no longer clearly distinguishable. Moreover, the full vowel frequency spectrum shows the amplitudes of different harmonics to be more comparable, as all the energies of different vowel harmonics are summed into the same harmonic peaks.

On the other hand, zooming into the Czech consonant 'z', a harsher 's', the time-domain response and the frequency spectrum is more akin to noise, as visible on the plots below:





This “noise-like” quality of the consonants is a consequence of how the sound is produced, as for a lot of consonants, air is forced through a narrow constriction in the throat and through the teeth. This causes turbulent and irregular airflow, unlike the periodic vibrations of the vocal cords responsible for vowels. Generally, we observed that consonants manifest themselves in higher frequencies compared to vowels, typically in the range of 2 kHz - 10 kHz, especially for fricatives like ‘s’. This hypothesis is supported in the 5cm and 1m frequency spectra by the rapidly decaying amplitude of the frequency components past about ~10 kHz. While the 10 kHz – 20 kHz range still contains components responsible for the voice sounding “fuller”, 10+ kHz range is often attenuated in EQ for voice enhancement.

Overall, we found the region between 100 Hz and 2 kHz to be dominated primarily by the sharp peaks of vowels with the fundamental and its harmonics, and the 2 kHz – 10 kHz region to be dominated by a broadband spectrum of higher frequencies responsible for consonants. Furthermore, the region between 0 Hz and 80 Hz rarely contains the human voice and is dominated by “rumble” noise or mains noise. This region is usually heavily attenuated in post-processing. In addition, we found that speech energy generally became much weaker than 10+ kHz, with that region being dominated by microphone hiss and other environmental noise.

If we were to compare the frequency composition of the 1m recording with the 5cm recording, the harmonics remain consistent in frequency throughout but differ in relative amplitudes. The 1m voice presents higher attenuation, as visible on the y-axis dB scale, and the spectrum is more likely to be affected by noise and room reflections. Moreover, the 5cm recording constitutes more bass (low) frequency components, which we attribute mostly to plosive consonants, such as ‘p’, ‘b’, or ‘t’, producing bursts of air in the near proximity and creating low frequencies below 200Hz. On the other hand, sounds like ‘s’ or ‘sh’ affect the high frequency ranges above 2kHz, which are also more prominent in the 5cm recording.

Part 2 - Linear Audio Enhancement

Continuing, our approach to the second part of the assignment was to use our knowledge from part 1 about the frequency composition of Daniel’s voice, and try to linearly enhance, attenuate, and filter different frequency components and bands. To test our hypothesis, the mixing desk in Rankine 509 Lab was used to physically check what frequencies make a difference in Daniel’s voice. Furthermore, the SM58 microphone’s frequency response was utilized as a guiding principle throughout our testing as a battle-tested frequency standard of linear enhancement.

It was discovered that it was most beneficial to the overall sound of the voice to boost the fundamental vowel frequency 100Hz and some of its harmonics, such as 300Hz. Furthermore, the 5 cm recording was often close to clipping due to the present of loud plosives, likely a result of the proximity to a directional microphone, meaning our linear enhancement had to be tuned correctly to avoid amplifying the plosives further.

Contrarily, in the 1 m recording moderate low frequency boosting was utilized to restore some warmth to the voice, as the bass decayed more strongly with distance than other frequencies. Moreover, other harmonics, including the 200Hz or 600Hz in fact made the sound worse when

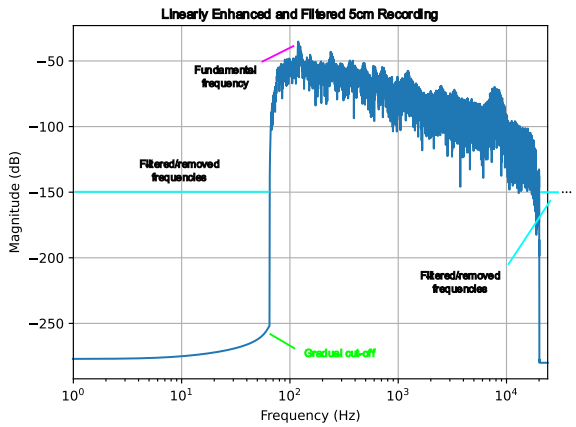
enhanced, so they were either left constant or attenuated by about -5dB, which was the case for the 600Hz and higher harmonics. This attenuation was not only audibly clear in Rankine 509 but also seen on the SM58 frequency response graph.

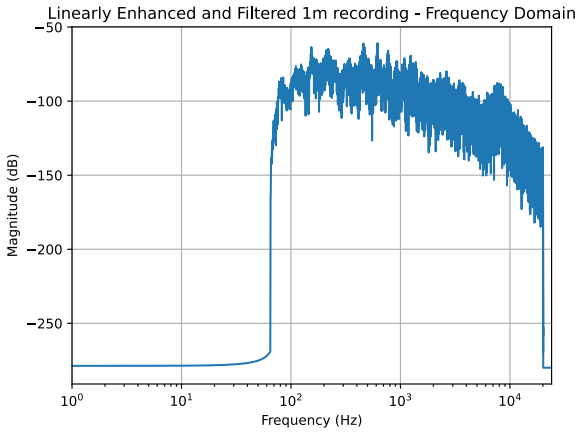
Furthermore, boosting the 8kHz frequency band by around 4-5 dB made significant difference in the perceived voice clarity and brightness, since male voices are very bass-heavy, and enhancing high frequency harmonic components like this can help balance the voice, and make the individual sounds more distinct and words more distinguishable. This is because, as was already mentioned in part 1, these high frequencies form the consonant sounds, including ‘s’ and ‘sh’. The exact overall settings differ between the 5 cm and 1 m recordings, but both were tuned to achieve a better sounding result.

Finally, we also decided to remove the very low and very high frequency components that we attributed to noise bands. For example, the range of human hearing is 20Hz-20kHz, with peak sensitivity around 2-5kHz, as we applied a bandpass filter of such bandwidth matching the range of human hearing. Another frequency band that was filtered out was the mains noise centered around 50Hz, and overall, any “rumble” below ~80 Hz, as even Daniels’ voice has little to no presence in the sub-100 Hz region. Our after-filter bandwidth was therefore ~80 Hz – 20 kHz, and such filtering was enough as not much noise was present during recording.

The plot below displays the final linearly enhanced and filtered frequency composition of Daniel’s voice recording. Gaussian filtering was applied to the recording for part 2 after improvements were discovered in part 3 due to iterative refinements made to the aural exciter. This is evident in the gradual cut-off of frequencies as opposed to a sharp transition. Since it is a log scale, the slower cut-off is more evident in the low-frequency filtering. If this were a linear scale, both would be visibly gradual.

It is apparent that the fundamental frequency remains at 100-120Hz, and the harmonics have changed energies appropriately to the enhancements we made, with a significant peak at 6-8kHz as well, in line with our intentions. As previously discussed in part 1, for the 1m recording the gain of the lower frequency harmonics was significantly increased compared to that of the 5cm voice, due to the increased decay of bass with distance than the mid or high bands. Below are the full filtered, and enhanced, spectra for the 5 cm and 1 m recordings.

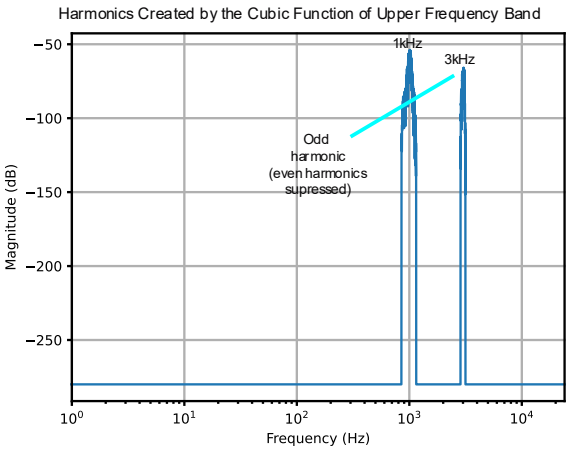




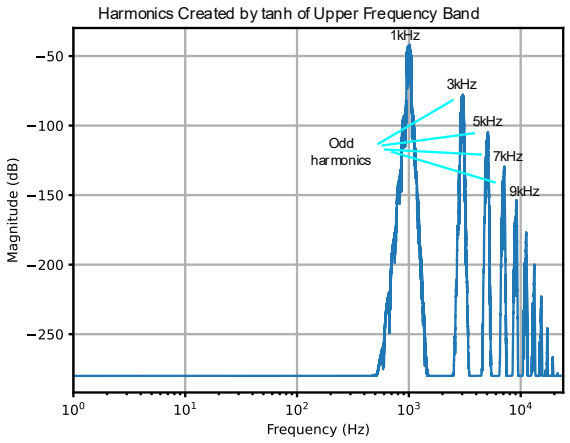
Part 3 - Non-linear Aural Exciter

The third part of this assignment concerns the use of non-linear methods of audio enhancement, specifically the “Aural Exciter” method. In this method, two distinct bands of the original audio are isolated, a lower and an upper band. A non-linear function is then passed over the bands to create harmonics, which are then mixed back into the audio, leading to a better-sounding voice.

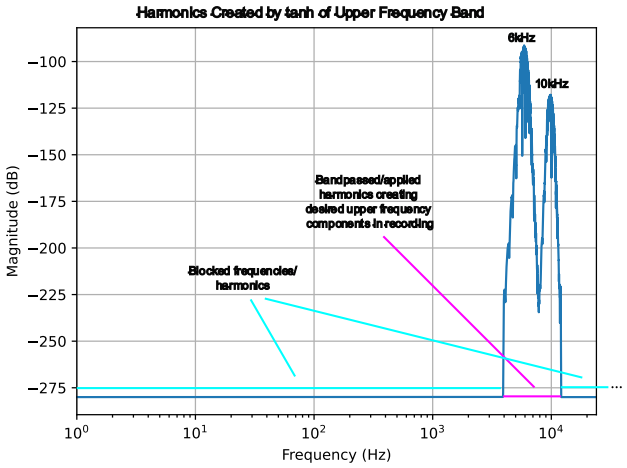
Initially, we used a simple cubic function, $y = x^3$, to generate the harmonics. While this worked well, we observed that the cubic function generates a disproportionaly strong third harmonic, which was unsuatable for our intentions, as can be observed on the plot below:



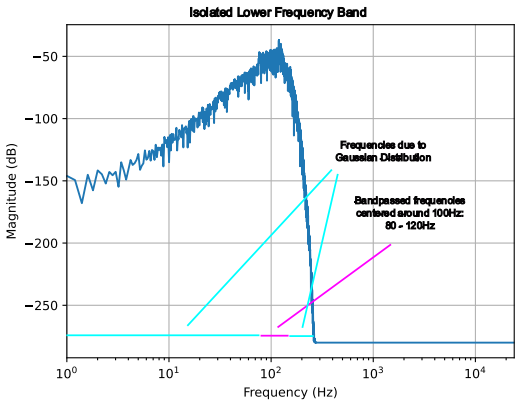
The increased amplitude of the higher frequency harmonics compared to the lower frequency harmonics resonated poorly with Daniel’s lower voice. Thus, our first improvement came from switching to a more natural hyperbolic tangent, tanh(), function. While both functions suppress the even harmonics and only generate odd harmonics, we observed that tanh() resulted in not only more consistent harmonics generation but also a more natural decay of the harmonics’ energies, contributing to a better sounding voice, as is visible on the plot below, leading to a more natural sounding addition to the original recording:

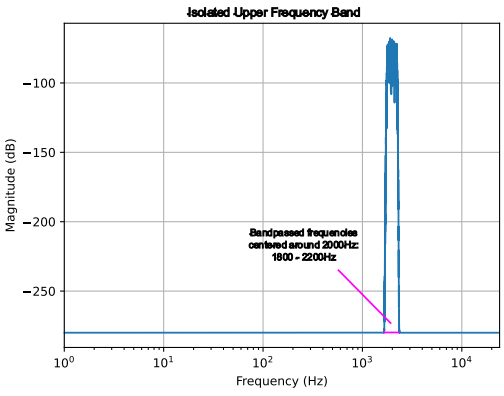


Secondly, the harmonics created by the tanh() function span a large amount of higher frequencies, thus we required a way to separate the useful, better-sounding harmonics from those contributing only noise, hiss and ringing. Therefore, our second addition to the aural exciter was a bandpass filter applied after the time-domain harmonic generation. This allowed us to tune easily the generated harmonics to separate the useful ones from the noise-contributing ones. The plot below illustrates harmonics generated by tanh() and bandpassed to allow only the first couple harmonics.



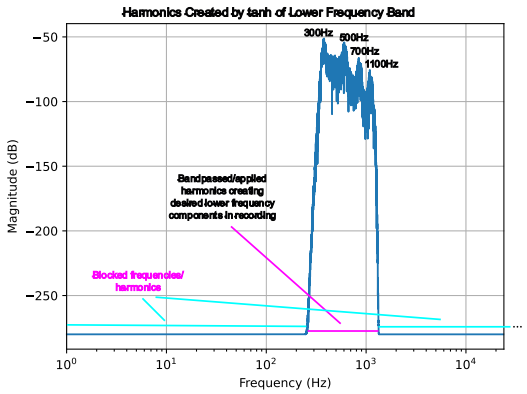
Thirdly, with the addition of the bandpass filter following the harmonic generation, the aural exciter now required two distinct FFT passes, one to separate the core upper and lower bands from the original audio and one to clean the generated harmonics. While this approach would work without issue in a fully analogue system using LRC filters, in digital post-processing applying a simple 1/0 mask to the spectrum creates sharp vertical edges. In turn, when applying the inverse FFT function to return to the time-domain, these edges result in unwanted high frequency components being introduced into the signal. In some cases during our testing, the two FFT passes with sharp filter edges caused audible time-domain ringing in the final signal. Therefore, we rewrote the bandpass filter to use a Gaussian mask with a tunable sigma value instead, allowing for smooth edges tunable in length. This change stopped the ringing from occurring, and was also retrospectively applied to the bandpass filters used in step 2. Below is an example of the lower and upper frequency pre-harmonicized bands with a large sigma value, exhibiting gradual cut-off rather than sharp edges:





Subsequently, we noticed the final result of the aural exciter sometimes resulted in peaking and overall higher volume. In order to preserve the intended volume of the voice, in other words making sure whispers stay whispers and yells stay yells, we applied RMS normalization to the aural exciter. In our approach, before applying any changes to the audio we first calculate the RMS amplitude. Then, after we generate, filter and reintroduce the harmonics, the final fused audio's RMS is compared to the initial RMS. If the two don't match, we alter the final audio so as to match the RMS amplitude. This also allowed for easier comparison between the original and aurally excited audio during parameter tuning, as any change in perceived loudness or clarity was due to the introduced harmonics and not due to a simple increase in total gain.

Lastly, while the aural exciter now resembled the diagram on the assignment sheet, we lacked granularity of parameters during tuning. As of this stage, we simply mixed the generated lower and upper harmonics back with the original audio in equal parts. As an improvement, we added a "mixer gain" to each harmonic addition, allowing full control into how much of each harmonic generation we're adding into the final signal. Furthermore, we added a similar "harmonic gain" parameter to both frequency bands, which was added into the tanh() function and thus controlled the strength of the generated harmonics. Below is a plot of the lower frequency band with generated harmonics, applied Gaussian filter and the harmonic gain parameter:

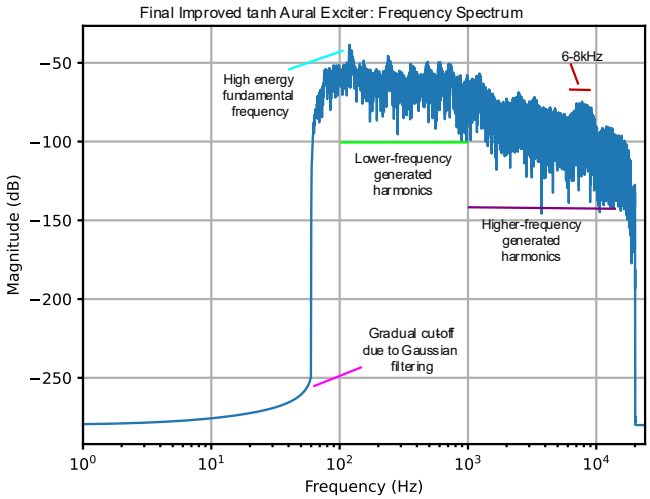


Using the added tunable parameters, we aimed for a slight addition of the lower band to aid the male lower voice and especially aimed to add harmonics in the 6-8 kHz region, as this addition was most notable during the Rankine 609 testing. At first our bandpass separators were too narrow around 10 Hz for the lower and 40 Hz for the upper band, which led to unnatural sounds in the final mixing. We gradually increased both bandwidths to ensure more of the harmonic passed through, while making sure not to increase them too much to limit generating harmonics from noise.

Furthermore, we focused on tuning the upper bandpass frequency to ensure the first generated harmonic, or the third harmonic of the signal, was present around the 6-8 kHz range, where the human hearing is most sensitive. After various testing we found a sweetspot of around 2300 Hz, whose third harmonics occurs around 7 kHz, precisely in the correct range. For the lower band, we found that amplifying the second harmonic, 240 Hz, in the original recording too much caused a "walkie-talkie" sounding and unpleasant voice, which was also discussed in the lab that most people don't prefer amplifying the second harmonic.

Thus we set the center frequency around 120 Hz, Daniel's fundamental, which resulted in a natural, fuller sounding voice after harmonic generation.

Overall, our final aural exciter design implemented many features found on the simplified block diagram as well as including several additions. Our additions helped us simplify the tuning process, such as adding separate gains into the mixer and harmonics generation, as well as ensuring our digital filters did not introduce unwanted higher frequencies. In the end, we managed to achieve a warmer and more prominent voice when comparing to the original as well as simpler linear enhancement using this non-linear approach. Below is the annotated plot of the final aural exciter frequency spectrum.



Conclusion

In conclusion, this lab effectively illustrated the ability of the FFT to alter and improve pre-recorded audio. Although this process is not real-time, the combination of FFT and inverse FFT remained useful visual inspection of recorded audio in both the time-domain as well as dB-scale frequency domain.

Furthermore, the distinct frequency bins created using the FFT process allowed for easy linear audio enhancement in part 2, where we used separate gains on various frequency bins to either attenuate or amplify certain frequencies in our recordings. This led to a clearer voice in the 5 cm recording and was especially noticeable in the 1 m recording where the enhancement added back the bass frequencies which attenuated with the increased distance in the original.

Lastly, in part 3 we explored the aural exciter non-linear enhancement and used an iterative process to analyze the final .wav recording and the various frequency spectra to change and add features to progressively improve the aural exciter. Although our design may be improved with additional post-processing, such as a De-Esser to remove the "s" sounds or a Downward Expander to eliminate noise in the pauses, we arrived at a final voice enhancement we believed was much clearer than that of the original. We successfully managed to amplify the fundamentals without making the voice sound overly rumble-like, matched RMS between the original and enhanced and added extra harmonics especially into the 6-8 kHz region where the ear is most sensitive, resulting in a more prominent, radio-ready voice.

Appendix

Helper Functions:

```
# Required imports

from scipy.io import wavfile
import numpy as np
import matplotlib.pyplot as plt
import os

# Source file path
path_to_files = ''
path_to_output = 'Output_Files/'

# ================================
# === Essential Math Functions ====
# ================================

def rms(x):
    return np.sqrt(np.mean(x**2))

def extract_audio_from_file(name_of_file):
    """
    Extracts audio from a WAV file using " scipy" and returns it as an
array
    """

    # Use the " scipy" library to read the audio file
    samplerate, audio_raw = wavfile.read(path_to_files + name_of_file +
".wav")
    audio_array = np.array(audio_raw)
    audio_array = audio_array / 32768.0  # Normalize the audio array to
-1/1 (0 dB), use 2^15 = 32768 for 16-bit audio

    return audio_array, samplerate

def create_WAV_file(name_of_file, audio_array, samplerate):
    """
    Creates a WAV file from an audio array and saves it to the output
folder
    """

    # Convert [-1, 1] floating-point audio to 16-bit PCM
    audio_array = np.clip(audio_array, -1.0, 1.0)
    audio_int16 = (audio_array * 32767).astype(np.int16)

    # Save the audio array as a WAV file
    wavfile.write(path_to_output + name_of_file + ".wav", samplerate,
audio_int16)

def fft_of_audio(audio_array, samplerate):
```

```
    audio_fft = np.fft.fft(audio_array)
    N = len(audio_array)

    # Make empty freq. array of size N
    frequency_axis = np.empty(N)

    # Frequency resolution = Fs / N
    # Frequency = k * resolution

    # fill array with correct frequencies
    for i in range(N):
        frequency_axis[i] = i * (samplerate / N)

    return audio_fft, frequency_axis

def inverse_fft(audio_fft):
    """
    Converts an FFT back to the time domain using IFFT
    """

    # Conver back to time and take only real values, as real sound
isn't imaginary :) unlike my best friend :(
    audio_ifft = np.real(np.fft.ifft(audio_fft))

    return audio_ifft

def apply_hamming_window_function(input_array):
    """
    Generates a Hamming window function of length N
    """

    N = len(input_array) # Number of samples
    np.arange(N) # Array of integers from 0 to N-1

    # This is the Mathematical Definition of the Hamming Window
Function
    hamming_window = 0.54 - 0.46 * np.cos(2 * np.pi * np.arange(N) / (N
- 1))

    # Multiplication in the time domain is convolution in the frequency
domain
    windowed_input_array = input_array * hamming_window

    return windowed_input_array


# ============================
# === Plotting Functions ====
# ============================

def plot_time_domain(audio_array, samplerate, name_of_plot,
show_plot=True):
    """
    Plots the time domain of an audio signal using an array from
"scipy"
    """
```

```
    # Calculate the maximum amplitude of the audio signal (normalized
to -1/1 but still indicates the loudness for comparison)
    max_amplitude = np.max(np.abs(audio_array))
    print(f"Max amplitude: {max_amplitude}")

    # Remap X-axis into time
    # Start at 0, end at length of audio / samplerate, number of points
is audio array length
    time_axis = np.linspace(0, len(audio_array) / samplerate,
num=len(audio_array))

    # Plotting time domain
    plt.plot(time_axis, audio_array)
    plt.title(f'{name_of_plot} - Time Domain')
    plt.xlabel('Time (s)')
    plt.ylabel('Amplitude')
    plt.ylim(-1, 1)
    plt.grid()
    plt.savefig(path_to_output + name_of_plot + '_time_domain.svg') #
[:-4 gets rid of .wav]
    if show_plot:
        plt.show()
    plt.close()

def plot_frequency_domain(audio_fft, frequency_axis, samplerate,
name_of_plot, logascale=False, show_plot=True):
    """
Plots the frequency domain of an audio signal using an array from
"scipy"
   """

    # Convert y-axis to dB scale
    audio_fft_log = 20 * np.log10(np.abs(audio_fft / len(audio_fft)) +
0.00000000000001) # Encountered log of zero so offset slightly by a
very small number to avoid 0 division

    # Plotting frequency domain
    if logascale:
        plt.plot(frequency_axis, audio_fft_log)
        plt.xscale('log')
        plt.ylabel('Magnitude (dB)')
    else:
        plt.plot(frequency_axis, np.abs(audio_fft))
        plt.ylabel('Amplitude')
    plt.title(f'{name_of_plot} - Frequency Domain')
    plt.xlabel('Frequency (Hz)')

    # Cutt off the mirror frequencies (above Fs / 2)
    plt.xlim(1, samplerate / 2) # Log can't start at 0

    plt.grid()
    plt.savefig(path_to_output + name_of_plot +
'_frequency_domain.svg') # [:-4 gets rid of .wav]
    if show_plot:
        plt.show()
    plt.close()
```

```
#============================
#=== Filtering Functions ===
#============================


def bandpass_filter(audio_fft, frequency_axis, samplerate,
                    low_cutoff, high_cutoff, sigma=20):

    # Convert mirrored FFT frequencies into their positive equivalents
for the gaussian attenutation to pass over them correctly
    frequency = np.where(
        frequency_axis <= samplerate / 2,
        frequency_axis,
        samplerate - frequency_axis
    )

    # Distance from the passband (zero for frequencies inside)
    distance = np.maximum(low_cutoff - frequency, 0) +
np.maximum(frequency - high_cutoff, 0)

    # Gaussian roll-off outside the passband
    mask = np.exp(-0.5 * (distance / sigma)**2)

    # Apply the smooth frequency mask
    return audio_fft * mask


# Same as band pass, but the guassian mask is inverted instead
def bandstop_filter(audio_fft, frequency_axis, samplerate,
                    low_cutoff, high_cutoff, sigma=20):

    # Convert mirrored FFT frequencies into their positive equivalents
for the gaussian attenutation to pass over them correctly
    frequency = np.where(
        frequency_axis <= samplerate / 2,
        frequency_axis,
        samplerate - frequency_axis
    )

    # Distance from the stopband (zero for frequencies inside)
    distance = np.maximum(low_cutoff - frequency, 0) +
np.maximum(frequency - high_cutoff, 0)

    # Gaussian attenuation around the stopband
    mask = np.exp(-0.5 * (distance / sigma)**2)

    # Invert the mask to reject rather than pass the selected
frequencies
    return audio_fft * (1 - mask)



def band_multiplier(audio_fft, frequency_axis, samplerate,
```

```
                    low_cutoff, high_cutoff, dB_gain, sigma=20):
    """
    Smoothly boosts or attenuates a frequency band and its mirrored FFT
band.
    Low sigma means sharper transitions
    """

    # Convert dB gain to linear multiplication
    multiplier = 10 ** (dB_gain / 20)

    # Generate Gaussian mask using the existing bandpass filter
    mask = bandpass_filter(
        np.ones_like(frequency_axis),
        frequency_axis,
        samplerate,
        low_cutoff,
        high_cutoff,
        sigma # Low sigma means sharper transitions and vice versa
    )

    # Smoothly apply the gain according to the Gaussian mask
    audio_fft_multiplied = audio_fft * (1 + (multiplier - 1) * mask)

    return audio_fft_multiplied
```

Part 1 - Graphing

```
def main():

    # Make sure the proper output path is used
    global path_to_output

    for current_audio_name in ["original_speech_5cm",
"original_speech_1m"]:

        show_plots = True

        # Set output folder for the current recording
        path_to_output = f"Output_Files/{current_audio_name}/"
        os.makedirs(path_to_output, exist_ok=True)

        #==================================
        #   ===   PART ONE - PLOTTING   ===
        #==================================

        # Extract the time domain and frequency domain from the WAV
        raw_audio, samplerate =
extract_audio_from_file(current_audio_name)

        # Use the window function to smooth the edges before plotting
the FFT (shows better frequency domain)
        windowed_audio = apply_hamming_window_function(raw_audio)
        windowed_fft, frequency_axis = fft_of_audio(windowed_audio,
samplerate)

        # Original signal plotted for analysis
        plot_time_domain(raw_audio, samplerate, current_audio_name +
"_02_original_waveform", show_plot=show_plots)
        plot_frequency_domain(windowed_fft, frequency_axis, samplerate,
current_audio_name + "_01_original_spectrum_hamming_windowed",
logascale=True, show_plot=show_plots)

if __name__ == "__main__":
    main()
```

Part 2 – Linear Enhancement

```
def main():

    show_plots = True

    # Make sure the proper output path is used
    global path_to_output

    # ========= ====================================
    # ========== 5cm audio enhancement =============
    # ========= ====================================

    current_audio_name = "original_speech_5cm"

    # Set output folder for the current recording
    path_to_output = f"Output_Files/{current_audio_name}/"
    os.makedirs(path_to_output, exist_ok=True)

    # Extract the time domain and frequency domain from the WAV
    raw_audio, samplerate = extract_audio_from_file(current_audio_name)
    fft_audio, frequency_axis = fft_of_audio(raw_audio, samplerate)

    # ====== Settings found from the Mixer in the DSP Rankine Lab
=======
    # ===== Combined with settings inspired by the SM58 microphone
======

    # Low-frequency bass
    fft_audio = band_multiplier(fft_audio, frequency_axis, samplerate,
90, 110, dB_gain=1, sigma=20)
    fft_audio = band_multiplier(fft_audio, frequency_axis, samplerate,
190, 210, dB_gain=-1, sigma=20)
    fft_audio = band_multiplier(fft_audio, frequency_axis, samplerate,
290, 310, dB_gain=3, sigma=50)
    fft_audio = band_multiplier(fft_audio, frequency_axis, samplerate,
390, 410, dB_gain=3, sigma=50)

    # Reduce boxiness without removing too much -> high attenuation
leads to "dull" voice
    fft_audio = band_multiplier(fft_audio, frequency_axis, samplerate,
700, 900, dB_gain=-3, sigma=50)

    # High-frequency presence boost -> resonanted quite well with
Daniel's voice
    fft_audio = band_multiplier(fft_audio, frequency_axis, samplerate,
3000, 4500, dB_gain=3, sigma=250)
    fft_audio = band_multiplier(fft_audio, frequency_axis, samplerate,
7500, 8500, dB_gain=4, sigma=200)



    # High-frequency roll-off -> higher sigma to be smoother
    fft_audio = band_multiplier(fft_audio, frequency_axis, samplerate,
11000, 13000, dB_gain=-3, sigma=500)
    fft_audio = band_multiplier(fft_audio, frequency_axis, samplerate,
13500, 20000, dB_gain=-5, sigma=500)
```

```
    # Noise filtering
    filtered_fft = bandpass_filter(fft_audio, frequency_axis,
samplerate, 20, 20000, sigma=20) # 20+ kHz Inaudible to humans anyway
    filtered_fft = bandstop_filter(filtered_fft, frequency_axis,
samplerate, 0, 70, sigma=10) # Removes most background "hum", human
voice isn't present there anyway

    # Inverse FFT
    inverse_filtered_audio = inverse_fft(filtered_fft)

    # Export
    create_WAV_file("enhanced_speech_5cm", inverse_filtered_audio,
samplerate)

    # ========== Plotting ============

    # Apply Hamming window FOR PLOTTING ONLY -> this does not affect
the actualy audio
    windowed_filtered_audio =
apply_hamming_window_function(inverse_filtered_audio)
    windowed_filtered_fft, frequency_axis =
fft_of_audio(windowed_filtered_audio, samplerate)

    # Plot the windowed spectrum
    plot_frequency_domain(windowed_filtered_fft, frequency_axis,
samplerate, "enhanced_speech_5cm_linear_filtered_spectrum",
logascale=True, show_plot=show_plots)




    #
================================================================================
================================================================================
==============




    # ============================================
    # ========= 1m audio enhancement =============
    # ============================================

    current_audio_name = "original_speech_1m"

    # Set output folder for the current recording
    path_to_output = f"Output_Files/{current_audio_name}/"
    os.makedirs(path_to_output, exist_ok=True)

    # Extract the time domain and frequency domain from the WAV
    raw_audio, samplerate = extract_audio_from_file(current_audio_name)
    fft_audio, frequency_axis = fft_of_audio(raw_audio, samplerate)
```

```
    # ====== Settings found from the Mixer in the DSP Rankine Lab
=======
    # ===== Combined with settings inspired by the SM58 microphone
======

    # Low-frequency bass - preserve natural vocal body
    fft_audio = band_multiplier(fft_audio, frequency_axis, samplerate,
70, 100, dB_gain=5, sigma=20)
    fft_audio = band_multiplier(fft_audio, frequency_axis, samplerate,
190, 210, dB_gain=5, sigma=20)
    fft_audio = band_multiplier(fft_audio, frequency_axis, samplerate,
250, 450, dB_gain=5, sigma=80)

    # Gentle reduction of boxiness
    fft_audio = band_multiplier(fft_audio, frequency_axis, samplerate,
600, 800, dB_gain=-1, sigma=100)

    # Gentle presence without excessive sibilance
    fft_audio = band_multiplier(fft_audio, frequency_axis, samplerate,
2500, 4500, dB_gain=1.5, sigma=300)
    fft_audio = band_multiplier(fft_audio, frequency_axis, samplerate,
7500, 8500, dB_gain=1, sigma=300)

    # Gentle high-frequency noise reduction
    fft_audio = band_multiplier(fft_audio, frequency_axis, samplerate,
13500, 20000, dB_gain=-3, sigma=500)

    # Noise filtering
    filtered_fft = bandpass_filter(fft_audio, frequency_axis,
samplerate, 20, 20000, sigma=20)
    filtered_fft = bandstop_filter(filtered_fft, frequency_axis,
samplerate, 0, 65, sigma=10)

    # Inverse FFT
    inverse_filtered_audio = inverse_fft(filtered_fft)

    # Export
    create_WAV_file("enhanced_speech_1m", inverse_filtered_audio,
samplerate)

    # ============ Plotting ============

    # Apply Hamming window FOR PLOTTING ONLY -> this does not affect
the actualy audio
    windowed_filtered_audio =
apply_hamming_window_function(inverse_filtered_audio)
    windowed_filtered_fft, frequency_axis =
fft_of_audio(windowed_filtered_audio, samplerate)

    # Plot the windowed spectrum
    plot_frequency_domain(windowed_filtered_fft, frequency_axis,
samplerate, "Linearly Enhanced and Filtered 1m recording",
logascale=True, show_plot=show_plots)
if __name__ == "__main__":
    main()
```

Part 3 – Non-Linear Aural Exciter

```
#============================
#    === Aural Exciter ===
#============================

# The function definition is really long to allow for useful tweaking
of the aural exciter parameters.

def aural_exciter(
    audio_raw, samplerate, name_of_audio,
    high_fundamental_frequency, low_fundamental_frequency,
    band_gap_low_frequency, band_gap_high_frequency,
    high_harmonic_low_cutoff, high_harmonic_high_cutoff,
    low_harmonic_low_cutoff, low_harmonic_high_cutoff,
    upper_band_harmonic_gain=10, lower_band_harmonic_gain=10,
    lower_band_mixer_gain=0.5, upper_band_mixer_gain=0.5,
    show_plots = True
):
    """
    Enhances the voice in an audio signal using harmonic addition
    """

    # Check the RMS of the original audio and the two bands
    original_rms = rms(audio_raw)
    print("Original RMS before exciter:", original_rms)

    # Make the FFT
    audio_fft, frequency_axis = fft_of_audio(audio_raw, samplerate)

    # ============== APPLYING LOW AND HIGH-PASS FILTERS ============

    # First, isolate the two frequency bands
    high_frequency_band = bandpass_filter(audio_fft, frequency_axis,
samplerate, high_fundamental_frequency - band_gap_high_frequency,
high_fundamental_frequency + band_gap_high_frequency)
    low_frequency_band = bandpass_filter(audio_fft, frequency_axis,
samplerate, low_fundamental_frequency - band_gap_low_frequency,
low_fundamental_frequency + band_gap_low_frequency)

    # Then, plot the frequency domain of the two bands
    plot_frequency_domain(high_frequency_band, frequency_axis,
samplerate, name_of_audio + "_04_upper_input_band", logascale=True,
show_plot=show_plots)
    plot_frequency_domain(low_frequency_band, frequency_axis,
samplerate, name_of_audio + "_05_lower_input_band", logascale=True,
show_plot=show_plots)

    # Then, convert back to time domain to apply the non-linearity
    high_frequency_band_time = inverse_fft(high_frequency_band)
    low_frequency_band_time = inverse_fft(low_frequency_band)

    # ============ GENERATING EXTRA HARMONICS ===============

    # Non-linear fuctions, including a gain parameter, are used to
generate extra harmonics from the isolated bands
```

```
    # OPTION 1
    # HYPERBOLIC TANGENT

    high_frequency_band_time_harmonics =
np.tanh(upper_band_harmonic_gain * high_frequency_band_time)
    low_frequency_band_time_harmonics =
np.tanh(lower_band_harmonic_gain * low_frequency_band_time)

    # OPTION 2
    # CUBE FUNCTION
    """
    high_frequency_band_time_harmonics =
np.power(upper_band_harmonic_gain * high_frequency_band_time, 3)
    low_frequency_band_time_harmonics =
np.power(lower_band_harmonic_gain * low_frequency_band_time, 3)
    """
    # ================ FILTERING THE EXTRA HARMONICS ================

    # Then, convert back to frequency domain to filter the harmonics
    high_fft, frequency_axis =
fft_of_audio(high_frequency_band_time_harmonics, samplerate)
    low_fft, frequency_axis =
fft_of_audio(low_frequency_band_time_harmonics, samplerate)

    # Select useful harmonic regions
    high_fft = bandpass_filter(high_fft, frequency_axis, samplerate,
high_harmonic_low_cutoff, high_harmonic_high_cutoff)
    low_fft = bandpass_filter(low_fft, frequency_axis, samplerate,
low_harmonic_low_cutoff, low_harmonic_high_cutoff)

    # Check that harmonics are created and filtered
    plot_frequency_domain(high_fft, frequency_axis, samplerate,
"_06_higher_band_harmonics", logascale=True, show_plot=show_plots)
    plot_frequency_domain(low_fft, frequency_axis, samplerate,
"_07_lower_band_harmonics", logascale=True, show_plot=show_plots)

    # Return to time domain
    high_frequency_band_time_harmonics = inverse_fft(high_fft)
    low_frequency_band_time_harmonics = inverse_fft(low_fft)

    # Check the RMS of generated, and filtered, harmonics
    print("Upper harmonic RMS:",
rms(high_frequency_band_time_harmonics))
    print("Lower harmonic RMS:",
rms(low_frequency_band_time_harmonics))

    # ========== FUSED AUDIO & RMS NORMALIZATION ============

    # Remove the unwanted frequencies from the original audio
    audio_fft = bandpass_filter(audio_fft, frequency_axis, samplerate,
20, 20000) # Humans only hear from 20 - 20,000 Hz
    audio_fft = bandstop_filter(audio_fft, frequency_axis, samplerate,
0, 60, sigma=10) # Remove the "hum" of the environment (lower sigma to
not eat away at the bass of the voice)
    filtered_audio = inverse_fft(audio_fft)

    # ===== Now, fuse all three together in the mixer =====
```

```
    fused_audio = filtered_audio + upper_band_mixer_gain *
high_frequency_band_time_harmonics + lower_band_mixer_gain *
low_frequency_band_time_harmonics

    # RMS of the fused audio
    fused_rms = rms(fused_audio)
    print("Fused audio RMS:", rms(fused_audio))

    # Normalize the fused audio to the original RMS to retain - >
WHISPERS = WHISPERS & YELLS = YELLS
    fused_audio = fused_audio * (original_rms / fused_rms)

    # ============== PLOTTING THE FINAL AUDIO ==============

    # Apply Hamming window FOR PLOTTING ONLY -> this does not affect
the actualy audio
    windowed_fused_audio = apply_hamming_window_function(fused_audio)
    fused_audio_fft, frequency_axis =
fft_of_audio(windowed_fused_audio, samplerate)

    plot_frequency_domain(fused_audio_fft, frequency_axis, samplerate,
name_of_audio + "_08_final_aural_excited_spectrum", logascale=True,
show_plot=show_plots)

    return fused_audio



# ================
# ===== MAIN =====
# ================

def main():

    show_plots = True
    global path_to_output

    # ==========================================
    # ========= 5cm audio enhancement ==========
    # ==========================================

    current_audio_name = "original_speech_5cm"

    path_to_output = f"Output_Files/{current_audio_name}/"
    os.makedirs(path_to_output, exist_ok=True)

    # Extract the time domain and frequency domain from the WAV
    raw_audio, samplerate = extract_audio_from_file(current_audio_name)

    excited_audio = aural_exciter(
        raw_audio,
        samplerate,
        current_audio_name,

        # Input frequency bands
        high_fundamental_frequency=2300,
        low_fundamental_frequency=120,
```

```
        band_gap_low_frequency=40,
        band_gap_high_frequency=50,

        # Output harmonic frequency bands
        high_harmonic_low_cutoff=6000,
        high_harmonic_high_cutoff=8000,
        low_harmonic_low_cutoff=250,
        low_harmonic_high_cutoff=850,

        # Harmonic generation and mixing
        upper_band_harmonic_gain=5,
        lower_band_harmonic_gain=6,
        lower_band_mixer_gain=0.6,
        upper_band_mixer_gain=0.5,

        show_plots=show_plots
    )
    create_WAV_file("exciter_speech", excited_audio, samplerate)

if __name__ == "__main__":
    main()
```